

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA1402

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 20

Total Marks:100

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I Al Majeed ¹⁹²⁴²⁴³¹⁵ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Explanation	25%				
Application	20%				
Organization	15%				
Accuracy	10%				
Clarity	5%				

Faculty Incharge

Slot-B

192424326

Annexure A

SHORT ANSWER TEST

1. Construct language for the following Regular Expression:2

$(a|b)^*a(a|b)|(a|b)$

2. a) Illustrate the predictive parser for the following grammar. 4

$S \rightarrow (L) | a$

$L \rightarrow L, S | S$

3. Analyse, is it possible, by modifying the grammar in any way to construct predictive parser for the language of

$S \rightarrow SS+ | SS^* | a$

String: "aa+a" 4

4. Eliminate left recursion from the grammar $S \rightarrow Sa|Sb|c$ 2
5. The following grammar contains common prefixes, making it unsuitable for predictive parsing.

$A \rightarrow abcD | abcE | f$

Apply left factoring to remove the common prefixes and rewrite the grammar.5

6. Consider the following statement:10

$Degree - F = Degree - C * 1.8 + 32$

Show the representation of this statement after each phase of the compiler and explain.

7. Consider the following C code fragment:4

`sum = 0;`

`for (i = 1; i < 5; i++)
sum = sum + i;`

Show the representation of the above code after each phase of the compiler.2

8. Consider the following grammar.

$S \rightarrow aABb, A \rightarrow c|\epsilon, B \rightarrow d|\epsilon$

Calculate FIRST and FOLLOW, Obtain the Predictive Parsing Table.
not. Show the parser moves for the input string **acdb**.

9. Write a simple Lex program to recognize and count identifiers in a given input.2
10. Write the rule to Eliminate Left Recursion and Left Factoring.2
11. What are the actions performed by a shift-reduce parser? Explain each action with a suitable example.3
12. Consider the following grammar:

$S \rightarrow pBB \rightarrow qB|r$

Find the leftmost derivation and construct the parse tree for the input string **pqq**.3

13. Consider Grammar: $E \rightarrow E+T | T$ $T \rightarrow T * F | F$ $F \rightarrow (E) | id$

Eliminate left recursion and find FIRST and FOLLOW 5

14. For the following grammar, determine whether the string **abcc** is accepted using shift-reduce parsing. Show all shift and reduce operations.

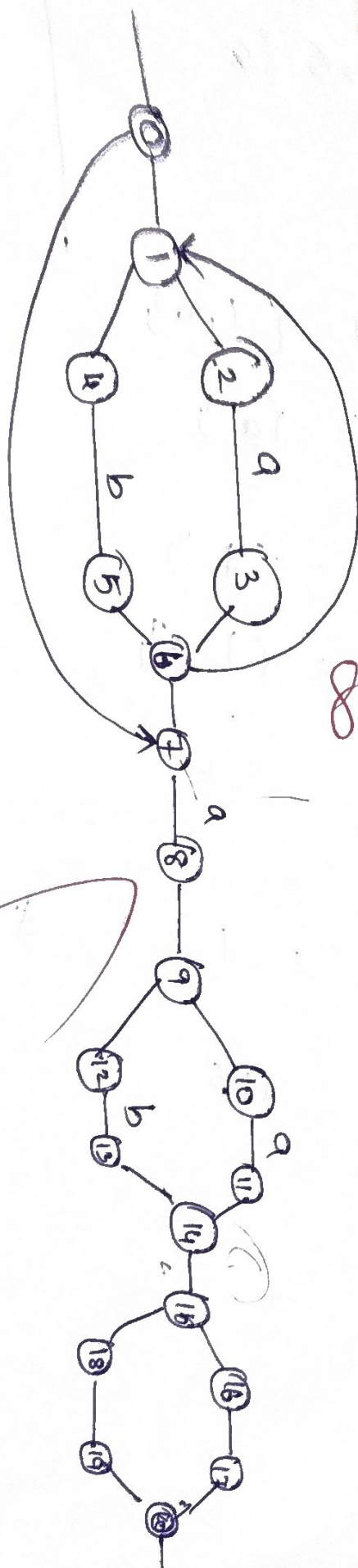
S'AI402 - Compiler Design

SLOT 5

CD2 AT-2

Short Answer Type

Construct NFA.



$(a|b)^* a (a|b) (a|b)$

82

Q. Predictive Parser:

$$S \rightarrow \cdot (L) \mid a$$

$$L \rightarrow L, S \mid a$$

$$S \rightarrow (L)S' \mid aS'$$

$$S' \rightarrow \epsilon$$

$$L \rightarrow aL' \mid \epsilon$$

$$L' \rightarrow \epsilon$$

$$\text{First}(S) = \{ (, a \}$$

$$\text{First}(L) = \{ a \}$$

$$\text{Follow}(S) = \{ \$ \}$$

$$\text{Follow}(L) = \{ (, \$ \}$$

$$\text{Follow}(L) = \text{First}(()$$

$$= ($$

$$\text{Follow}(() = \text{First}(\$,)$$

$$= (\$,)$$

Parsing table

NT					
	a	,	(	)	\$
S	(L)S'		(L)S'		
S'					
L	aL'				
L'					

$S \rightarrow SS+ | SS* | a$

String "aa+a"

$S \rightarrow SS+$

$S \rightarrow SS*$

$S \rightarrow a$

$S \rightarrow SS+S'$

$S' \rightarrow \epsilon$

$S' \rightarrow aS$

$\text{First}(S) = \{a\}$

$\text{First}(S') = \{a\}$

$\text{Follow}(S) = \{+, *, \$\}$

$\text{Follow}(S') = \{b, a, \$\}$

Parsing table:

$\backslash T$	+	*	a	\$
S	$S \rightarrow SS+$	$S \rightarrow SS*$	$S \rightarrow a$	
S'			$S \rightarrow aS$	

4.

$$S \rightarrow Sa \mid Sb \mid c$$

$$S \rightarrow Sa' \mid Sb' \mid c'$$

$$S \rightarrow \epsilon$$

$$S' \rightarrow \epsilon$$

5.

$$A \rightarrow \overset{a}{a} \overset{B}{b} \overset{c}{c} D \mid \underline{abc} E \mid \underline{\$}$$

$$A \rightarrow abcDA' \mid abcEA' \mid \underline{\$}A'$$

$$A' \rightarrow \epsilon$$

$$\text{First}(A) = \{a, \$\}$$

$$\text{Follow}(A) = \{ \$ \}$$

6.

~~$$\text{Degree-F} = \text{Degree-C} * 1.8 + 32$$~~

$$\text{Degree-F} = \text{Degree-C} * 1.8 + 32$$

Tokens :

Degree-F — Variable

= — Assignment operator

Degree-C — Variable

1.8 — float

+ — operator

Degree - P $\rightarrow$ id 1
 = $\rightarrow$ Operator (=)

Degree - C $\rightarrow$ id 2

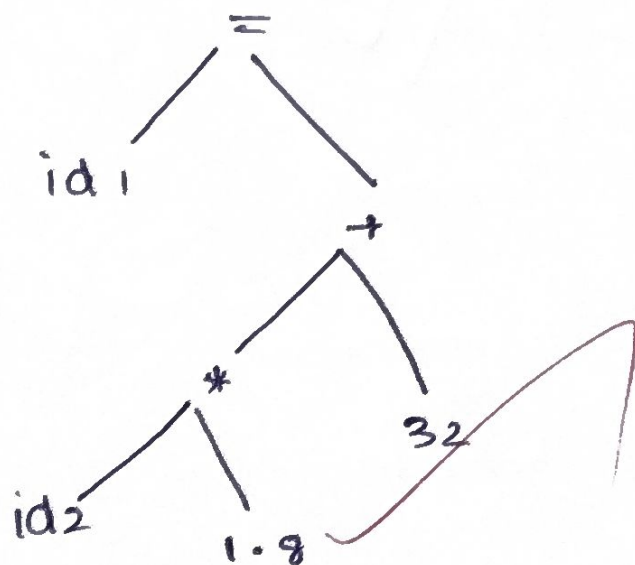
1.8 $\rightarrow$ id 3 ~~fact~~ (1.8)

+ $\rightarrow$ Operator (+)

32 $\rightarrow$ id 4, int (32)

1.8
 32
 1.8
 32
 1.8
 32
 1.8
 32

Parse tree :



Code generator :

id 3 = id 2 * 1.8

id 4 = id 3 + ~~id~~ 32

id1 = id 4

Code optimizer

$$id4 = id2 * 1.8 + 32$$

$$id1 = id4.$$

Final :

$$\text{Degree-F} = id4 * 1.8 + 32$$

$$\text{Degree-F} = id3 + 32$$

Assemble code:

MOV id2, 1.8

OPP id2, *

MOV id4, id3

MOV id3, 32

ADD id3, 32

10

Phases of Compiler

Sum = Sum + i

Lexical Analysis

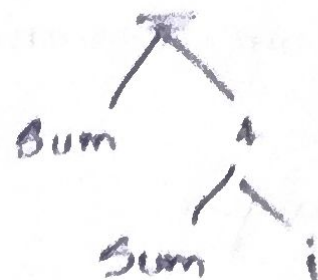
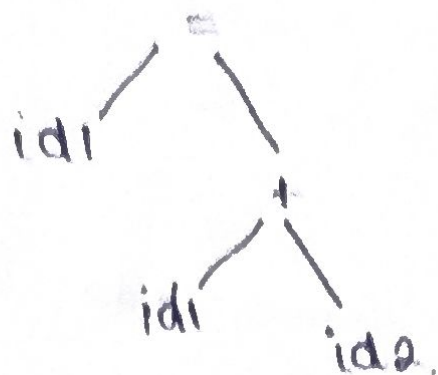
Separate the given source code into tokens and identify those corresponding lexemes.

Sum	→	Variable 1
=	→	Assignment operator
Sum	→	Variable.
+	→	Arithmetic operator
i	→	Variable 1

Sum	→	id 1
=	→	operator
Sum	→	id 1
+	→	operator
i	→	id 2.

Page 1900

20/11/20



Code generator :

$id3 = id1 + id2$

$id1 = id3$

$Sum = id1$

Code optimizer :

$id1 = id1 + id2$

Final

$Sum = id1 + id2$

ASSEMBLY CODE :

MOV (id, id2)

MOV (id3)

ADD (id1 + id2)

ADD (id3 + id1)

$S \rightarrow a \overset{1}{A} \overset{2}{B} b$, $A \rightarrow c | e$, $B \rightarrow d | e$

$S \rightarrow a A B b$
↑ ↑ ↑ ↑

$A \rightarrow c | e$

$B \rightarrow d | e$

First (S) = { a }

First (A) = { c, e }

First (B) = { d, e }

Follow (S) = { \$ }

Follow (A) = { d, \$ }

Follow (B) = { e, d, \$ }

4/10

S

a

c

d

4

~~S → A → B~~

A

A → X

B

B → D

{

/*

include <iostream> ~~using namespace~~

using namespace std;

String s;

Count = 0;

for (i = 1; i < 5; i++)

Count = Count + i;

if (s == '+' + "id" + "id" + "id" + "id" + "id",)

Count++;

else Count = 0;

Print f ("The count of identified", Count)

/*

*/

2

Rules to Eliminate Left recursion

$$A \rightarrow aA \mid b$$

$$B \rightarrow bB \mid a$$

- (1) In given all production take the first Production. (Consider the example)
- (2) To the first non-terminal production Apply (A') (A_{prime}) to the right hand side of each production

$$A \rightarrow aAA' \mid bA'$$

- (3) if there is Any same term or a non terminal and its prime then the production becomes ϵ Null set.

$$AA' \rightarrow \epsilon$$

- (4) Final Production after left recursion will be like $A \rightarrow bA' \mid \epsilon$

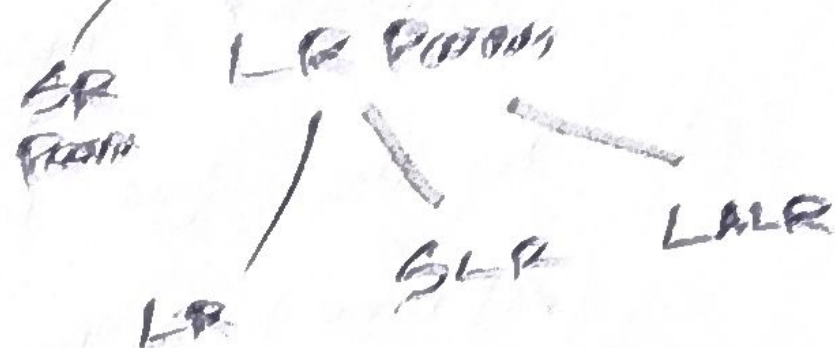
Shift Reduce Parser:

* This is a bottom up Approach, Where It will do, ~~shift~~ or Reduce at last Accept.

Top Down
Parser Approach



Bottom Up
Parser



REDUCE → It transforms the given input string
to the given production

SHIFT → It shift the identifier from the input
buffer if there is no match.

ACCEPT → If both Right hand side and the
input buffer is same i.e. ($\$ - \$$)
Then it is considered as Accept.

2. Consider the following grammar.

$$S' \rightarrow PB$$

$$B \rightarrow aB \mid \epsilon$$

$$S \rightarrow P B S'$$

$$S \rightarrow \epsilon$$

$$D \rightarrow qBB' | VB'$$

$$B \rightarrow VB' | \epsilon$$

$$Pqav \rightarrow id1id2 id2 id3$$

$$S \rightarrow PB$$

$$S \rightarrow PQB$$

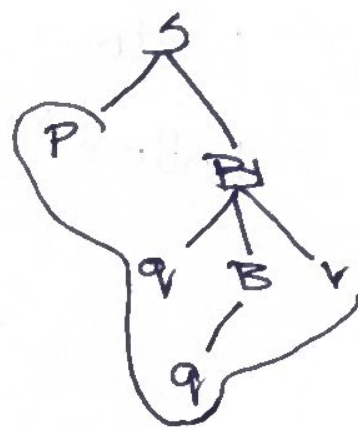
$$S \rightarrow PqavB$$

$$S \rightarrow Pqav$$

$$[B \rightarrow qB]$$

$$[B \rightarrow qB]$$

$$[B \rightarrow v]$$



$$S \rightarrow Pqav$$

3

$$E \rightarrow E+T | T$$

$$T \rightarrow T*F | F$$

$$F \rightarrow (E) | id$$

$$E \rightarrow E+TE | T$$

$$E' \rightarrow TE'$$

$$T \rightarrow T*F | F$$

$$T' \rightarrow FT'$$

$$F' \rightarrow (E) | id$$

$$~~F \rightarrow (E) | id~~$$

$$First(E) = \{ (, + \}$$

$$First(E') = \{ *, (, id \}$$

$$First(T) = \{ *, (, id \}$$

$$First(T') = \{ (, id \}$$

$$First(F) = \{ (, id \}$$

Follow (E) = { \$, (, * }

Follow (E') = { \$, ~~(~~, id, + }

Follow (T) = { \$, (, * }

Follow (T') = { \$, (, * }

Follow (F) = { \$, (, id, + }

14.

$S \rightarrow AB \mid A \rightarrow aA \mid a \mid B \rightarrow CB \mid c$

\$ S	abcc	
\$ AB	abcc	Reduce $[A \rightarrow AB]$
\$ aB	abcc	Reduce $[A \rightarrow a]$
\$ B	bcc	Some Cancellation
\$ CB	bcc	$[B \rightarrow CB]$
\$ CCB	bcc	$[B \rightarrow CB]$

Input not accepted because there is no 'b' in the given production

16

17
18
19

$S \rightarrow aAB$
 $A \rightarrow bA|b$
 $B \rightarrow c$

abbc

\$ S

\$ aAB

\$ AB

\$ bAB

\$ AB

\$ bB

\$ B

\$ c

\$

abbc\$

abbc\$

bbc\$

bbc\$

bc\$

bc\$

c\$

c\$

\$

Reduce [$b \rightarrow aAB$]

Cancelation

$A \rightarrow bA$

Cancelation

$A \rightarrow b$

Cancelation

$B \rightarrow c$

Accepted.

$$S' \rightarrow S$$

$$S \rightarrow aAB$$

$$A \rightarrow bA \mid c$$

$$B \rightarrow d$$

I_0 :

$$S' \rightarrow \cdot S$$

$$S \rightarrow \cdot aAB$$

$$A \rightarrow \cdot bA \mid \cdot c$$

$$B \rightarrow \cdot d$$

$I_{0,1}$

17.

$$S \rightarrow iEtSs' \mid a$$

$$S' \rightarrow eS \mid \epsilon$$

$$E \rightarrow b$$

$$\text{First}(S) = \{a\}$$

$$\text{First}(S') = \{e\}$$

$$\text{First}(E) = \{b\}$$

$$\text{Follow}(S) = \{\$ \}$$

$$\text{Follow}(S') = \{\$ \}$$

$$\text{Follow}(E) = \{\$ \}$$

NT \ T	a	e	b
S	$S \rightarrow l e l S'$		
S'		$S' \rightarrow e S$	
E			$E \rightarrow b$

18.

4

$$A \rightarrow a A B \mid a A \mid a$$

$$B \rightarrow b B \mid b.$$

$$A \rightarrow a A' \mid \epsilon$$

$$A' \rightarrow \epsilon$$

$$B \rightarrow b B' \mid \epsilon$$

$$B' \rightarrow \epsilon$$

2

9.

$S \rightarrow \alpha \beta \gamma$
 ABC

$A \rightarrow a | \epsilon$

$B \rightarrow r | \epsilon$

$C \rightarrow b | \epsilon$

$\text{Follow}(B) = \text{First}(C)$

$= b$

$\text{Follow}(B) = b$

~~$S \rightarrow \alpha \beta \gamma$~~

$\text{First}(B) = \text{First}(A) = \{a\}$

$\text{First}(A) = \{a\}$

$\text{First}(B) = \{r\}$

$\text{First}(C) = \{b\}$

$\text{Follow}(S) = \{ \$ \}$

$\text{Follow}(A) = \{ \$ \}$

$\text{Follow}(B) = \{ b, \$ \}$

$\text{Follow}(C) = \{ b, \$ \}$

20.

$S \rightarrow \alpha \beta \gamma | \alpha \beta \gamma D$

$A \rightarrow a | b b D$

$B \rightarrow a | \epsilon$

$C \rightarrow b | \epsilon$

$D \rightarrow c | \epsilon$

$$\text{First}(S) = \{a, 1\}$$

$$\text{First}(A) = \{a\}$$

$$\text{First}(B) = \{a\}$$

$$\text{First}(C) = \{b\}$$

$$\text{First}(D) = \{c\}$$

$$\text{Follow}(S) = \{\$, \#, ab\}$$

$$\text{Follow}(A) = \{\$, \#, 1\}$$

$$\text{Follow}(B) = \{\$, \#, a\}$$

$$\text{Follow}(C) = \{\$, \#, a, 1\}$$

$$\text{Follow}(D) = \{\$, \#, a, 1\}$$

}
{

C02 A1-2

Short Answer Test

15. For the following grammar, verify whether the string $abba$ is valid using shift-reduce parsing. 10

$$S \rightarrow AHA \mid aA \mid b \mid c \mid d$$

16. Consider the following grammar. 10

$$S \rightarrow SS \mid aAA \mid bA \mid cB \mid d$$

- Construct the canonical collection of LR(0) items.
- Construct the SLR(1) parsing table.

17. Consider the following grammar.

$$S \rightarrow IRS \mid a, S \rightarrow eS \mid \epsilon, R \rightarrow b$$

Obtain the Predictive Parsing Table. 5

18. Do left factoring in the following grammar: 3

$$A \rightarrow aAB \mid aA \mid a$$

$$B \rightarrow bB \mid b$$

19. Construct the LL(1) Parsing Table for the following grammars. 7

$$S \rightarrow ABCA \rightarrow a \mid \epsilon B \rightarrow r \mid \epsilon C \rightarrow b \mid \epsilon$$

20. Construct the LL(1) Parsing Table for the following grammars. 7

$$S \rightarrow 1S \mid qABCA \rightarrow a \mid b \mid bDB \rightarrow a \mid \epsilon C \rightarrow b \mid \epsilon D \rightarrow c \mid \epsilon$$

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers

Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand